PROYECTO DE CREACIÓN DE CHATBOTS · SEGUNDO CORTE

# ViajesYa · Asistente de reservas

Un chatbot que conversa con los clientes, entiende qué viaje quieren y guarda sus reservas de forma segura.

Integrantes: [Nombre 1], [Nombre 2], [Nombre 3] · Repositorio: `github.com/[usuario]/[repositorio]`

# ¿Qué es este proyecto?

Es un **chatbot**: un programa que conversa con una persona por texto, como si fuera un asesor.

Lo creamos para **ViajesYa**, una agencia de viajes en Armenia, Colombia. La agencia vende paquetes a destinos con **fecha de salida fija** y **cupos limitados**. Antes, un cliente debía llamar o ir en persona para reservar. Ahora puede escribir en un chat y el sistema hace el trabajo.

### El negocio

ViajesYa organiza salidas grupales a destinos colombianos como Caño Cristales (La Macarena), el Nevado del Ruiz, Manizales y el Eje Cafetero.

### El problema

Reservar es lento y propenso a errores: fechas, cupos, cruces de viajes y datos del cliente deben validarse a mano. Un asesor humano no puede atender 24/7.

> La idea central: **el cliente conversa en lenguaje natural** (“quiero ir a la Macarena en diciembre”) y el sistema entiende, valida y registra la reserva sin que tenga que llenar formularios.

# ¿Qué tareas resuelve el chatbot?

| El cliente quiere… | El sistema responde… |
|---|---|
| Ver destinos y precios | Muestra el catálogo con duración y valor en pesos. |
| Saber qué salidas hay | Lista las fechas disponibles con cupos reales. |
| Reservar un viaje | Recoge los datos en varios mensajes y pide confirmación. |
| Cambiar su reserva | Modifica cliente, destino o fecha si hay cupo. |
| Cancelar | Libera el cupo y conserva el registro como cancelado. |
| Consultar una reserva | Con el código `PN-…` muestra todos los datos. |

## Límites conocidos

- No procesa pagos en línea: la reserva se paga en la agencia.
- No modifica el catálogo ni las fechas de salida: solo reserva sobre lo que ya existe.
- No reemplaza al asesor humano para casos especiales o grupos grandes.
- Si la API de Groq o la FastAPI no responden, el chat avisa y no inventa datos.

# Las cinco herramientas y su papel

Cada herramienta tiene una tarea clara. Si una falla, las demás no se mezclan: el sistema sabe a quién culpar y qué hacer.

### Gradio

**La pantalla.** Dibuja la ventana del chat en el navegador: caja de texto, botones Enviar, Confirmar, Descartar y Reiniciar.

### Python

**El cerebro que conecta.** Recibe el mensaje, decide a quién pedirle qué, guarda el historial y aplica las reglas.

### Groq (modelo de lenguaje)

**El que entiende y redacta.** Interpreta “quiero ir a la macarena” y luego escribe una respuesta amable. Nunca decide si la reserva se hace.

### FastAPI

**El que manda sobre los datos.** Valida cupos, detecta cruces de viajes y ejecuta crear / modificar / cancelar. Es la única fuente de verdad.

### SQLite

**El cuaderno donde se guarda todo.** Un archivo `reservas.db` conserva las reservas aunque se apague el programa. Al reiniciar, las filas siguen ahí.

> Regla de oro del proyecto: **el modelo de lenguaje NUNCA toca la base de datos**. Solo interpreta y redacta. La API es la que dice “sí hay cupo” o “no se puede”.

# Cómo se conectan las piezas

**Persona** → escribe en Gradio (navegador)

↓

**Python** (chatbot_memoria.py) recibe el mensaje y lo envía a Groq para *interpretar* la intención

↓

**Groq** devuelve un JSON: “esto es una *crear*, con servicio = caño_cristales, fecha = 2026-12-05, cliente = Laura”

↓

**Python** llama a la **FastAPI** para validar y, si el cliente confirma, ejecutar la operación

↓

**FastAPI** revisa cupos, cruces e integridad y guarda en **SQLite** ( `reservas.db` )

↓

**Python** le pide a **Groq** que redacte la respuesta con el resultado *real* y se la muestra a la persona en Gradio

Este orden garantiza que el chatbot **nunca invente una reserva**: solo confirma cuando la API respondió que se guardó.

# Los tres pasos de cada turno

### 1 · Interpretar

Groq lee el historial y el catálogo real de la API, y devuelve un JSON con la **intención**: acción, cliente, servicio, fecha y código.

Ejemplo: `{"accion":"crear","servicio":"caño_cristales","fecha":"2026-12-05","cliente":"Laura"}`

### 2 · Ejecutar

Python consulta la FastAPI: ¿hay cupo?, ¿el cliente ya tiene un viaje que se cruce?, ¿el servicio existe? Solo si todo está bien, guarda.

### 3 · Redactar

Groq vuelve a escribir, ahora **en lenguaje natural y cálido**, usando únicamente el resultado real del paso 2. Si Groq falla en este paso, Python muestra un texto simple pero correcto (fallback).

> La **confirmación** (crear / modificar / cancelar) la decide **Python** con palabras como “sí”, “dale”, “ok”; nunca el modelo. Así se evita que el LLM confirme una operación por error.

# Una reserva paso a paso

hola, quiero ir a la macarena en diciembre

¡Hola! Qué lindo plan 🌿 Tenemos salida a **Caño Cristales** el **5 de diciembre de 2026**, 4 días, por **$830.000 COP**. ¿A nombre de quién hago la reserva?

Laura Gómez

Perfecto, Laura. Te resumo: **Caño Cristales**, salida **5 dic 2026**, **$830.000 COP**, a nombre de **Laura Gómez**. Todavía no está reservado. ¿Confirmas? (sí / no)

sí

¡Listo, Laura! 🎉 Tu reserva quedó registrada con el código **PN-0007**. Guárdalo para consultarla, modificarla o cancelarla.

Lo que no se ve: Groq interpretó cada mensaje, Python validó cupo y cruce con la API, y la fila quedó guardada en `reservas.db`. El botón **Confirmar** de Gradio equivale a escribir “sí”.

# Disponibilidad, cupos y cruces

| Situación | Qué hace el sistema |
|---|---|
| Cada viaje tiene una **fecha de salida fija** | No se inventan fechas: solo se ofrecen las que existen en `negocio.json` y en `viajes.py`. |
| El viaje dura **N días** (ej. 4 días) | Ocupa el intervalo *[fecha_salida, fecha_regreso]*, no solo un día. |
| Cupos limitados | Cada reserva descuenta 1 cupo. Si el cupo llega a 0, la salida aparece como **[LLENO]**. |
| Reservas cerradas por fecha de cierre | Después de `cierre_reservas` ya no se acepta la salida, aunque haya cupo. |
| Un cliente con dos viajes que se solapan | La API responde **409** y el bot avisa antes de pedir confirmación. Salir el día siguiente al regreso sí se permite. |
| Operación sin confirmar | El bot muestra un resumen y pide “sí” o “no”. Nunca crea sin confirmación explícita. |

> Todas estas reglas viven en la **FastAPI**, no en el modelo. Por eso el chatbot no puede saltárselas aunque el cliente “se lo pida” al LLM.

# Dónde se guarda todo: SQLite

Las reservas se guardan en el archivo `Demos/Demostraciones_02_03/reservas.db`. La primera vez que arranca la API importa el histórico de `reservas.json`; después ese archivo ya no se usa.

### Tabla `reservas`

Campos principales: `codigo` (PN-…), `cliente`, `servicio`, `fecha`, `fecha_regreso`, `estado` (activa / cancelada).

### Comprobación rápida

```
python -m sqlite3 reservas.db
"SELECT codigo, cliente, servicio, fecha, estado FROM
reservas;"
```

## ¿Por qué SQLite y no un archivo JSON?

- **Atomicidad:** cupos y cruces se comprueban y guardan en una sola transacción. No hay dos clientes viendo el mismo cupo.
- **Persistencia:** al reiniciar la API, las filas siguen ahí. Un JSON podía corromperse o quedar desactualizado.
- **Consultas:** filtrar por cliente, servicio o estado es directo.

# Qué probamos y qué obtuvimos

| Prueba | Resultado esperado | Estado |
|---|---|---|
| Conversación en varios turnos | El bot conserva los datos ya aportados (cliente, servicio, fecha). | ✅ Funciona |
| Dos sesiones y reinicio | Cada pestaña del navegador tiene su propia conversación; reiniciar borra la charla, no las reservas. | ✅ Funciona |
| Crear / modificar / cancelar | Los cambios se ven en `reservas.db` con el código PN correspondiente. | ✅ Funciona |
| Intervalos ocupados | Se rechaza una reserva que se cruza con otra activa del mismo cliente (409). | ✅ Funciona |
| Cancelación y persistencia | Libera el cupo y la fila queda con estado `cancelada`. | ✅ Funciona |
| Errores (API apagada / modelo inválido) | El chat avisa y no cambia el historial; se puede reintentar. | ✅ Funciona |

Evidencias: `evidencia_clase07.txt` (registro de turnos), capturas en `docs/evidencias/` y los scripts `probar_viajes.py`, `probar_cruces.py` y `probar_sqlite.py`.

# Qué cambió frente a la versión con Flowise y JSON

| Aspecto | Versión anterior (Flowise + JSON) | Versión actual (Python + Groq + FastAPI + SQLite) |
|---|---|---|
| Orquestación | Nodos visuales en Flowise | Código Python explícito y versionado en Git |
| Reglas del negocio | Difíciles de versionar y auditar | En `consulta_llm2.py` y `main.py`, con tests |
| Persistencia | Archivo JSON (propenso a corrupción) | SQLite con transacciones ACID |
| Control del modelo | Prompt dentro de la plataforma | Dos prompts separados: intérprete (temp. 0) y redactor (temp. 0.5) |
| Confirmación | Dependía del flujo visual | La decide Python con palabras clave; el LLM no puede confirmar |
| Interfaz | La de Flowise | Gradio con botones Confirmar / Descartar / Reiniciar |
| Reproducibilidad | Depende de la plataforma | `requirements.txt` + `iniciar_api.bat` + `iniciar_gradio.bat` |

> La gran diferencia: ahora **el equipo es dueño del código**. Cualquier regla se puede leer, probar y modificar sin depender de una plataforma externa.

# Limitaciones y mejoras pendientes

## Limitaciones actuales

- No procesa pagos ni emite facturas.
- Depende de la disponibilidad de Groq (API externa).
- El catálogo y las fechas se editan a mano en archivos.
- No hay autenticación de usuarios: cualquiera con el código puede consultar una reserva.

## Mejoras propuestas

- Panel administrativo para editar catálogo y fechas.
- Autenticación por cédula o correo para consultar reservas.
- Integración con pasarela de pagos (PSE, tarjeta).
- Notificaciones por correo o WhatsApp al confirmar.

> Documentar las limitaciones también es parte del proyecto: decir qué **no** hace el sistema es tan importante como decir qué sí hace.

# Cómo ejecutar el proyecto (para alguien nuevo)

1. Descargar o clonar el repositorio de GitHub.
2. Doble clic en `iniciar_api.bat`. La primera vez crea el entorno virtual e instala dependencias. Dejar esa ventana abierta (FastAPI en `http://127.0.0.1:8011/docs`).
3. Doble clic en `iniciar_gradio.bat`. Escribir la API key de Groq (no se muestra en pantalla) y el ID del modelo (ej. `llama-3.1-8b-instant`).
4. Abrir `http://127.0.0.1:7860` en el navegador.
5. Probar: *"hola, quiero ir a la macarena en diciembre"*.

```
# Alternativa manual desde la carpeta del repo:
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
# Terminal 1:
.\.venv\Scripts\python.exe -m uvicorn main:app --reload --port 8011
# Terminal 2:
$env:CHATBOT_API_URL = "http://127.0.0.1:8011"
.\.venv\Scripts\python.exe app_gradio.py
```

> Todos los comandos, variables y puertos están en el `README.md` y en `LEEME_CHATBOT.md`. La API key nunca se guarda en archivos: se pide oculta o se lee de una variable de entorno.

# Lo que cada integrante debe poder explicar

| Pregunta | Respuesta esperada |
|---|---|
| ¿Quién decide si se crea una reserva: el modelo o el sistema? | El sistema (Python + FastAPI). El modelo solo interpreta y redacta. |
| ¿Dónde viven las reservas? | En `reservas.db`, un archivo SQLite que persiste al reiniciar. |
| ¿Qué pasa si dos clientes quieren el último cupo? | La transacción SQLite garantiza que solo uno lo obtenga; el otro recibe “LLENO”. |
| ¿Qué pasa si Groq falla? | El chat avisa, no modifica el historial y se puede reintentar. La reserva no se crea. |
| ¿Cómo se evita que un cliente tenga dos viajes cruzados? | La API consulta `/disponibilidad` y responde 409; el bot lo avisa antes de confirmar. |
| ¿Por qué hay dos prompts distintos? | Uno interpreta con temperatura 0 (preciso) y otro redacta con 0.5 (natural). |
| ¿Dónde se ve que el sistema funciona? | En `evidencia_clase07.txt`, capturas en `docs/evidencias/` y los scripts de prueba. |

CIERRE

# ViajesYa · un chatbot que conversa y no inventa

El proyecto demuestra que un chatbot puede ser **útil y confiable** cuando separa responsabilidades: el modelo entiende y redacta, la API decide, la base de datos guarda.

### Repositorio

`github.com/[usuario]/[repositorio]`

Incluye `README.md`, `LEEME_CHATBOT.md`, código, datos de prueba y evidencias.

### Documentación

`docs/Bitacora_Segundo_Corte.md` · `docs/Presentacion_Segundo_Corte.pdf` · `docs/evidencias/`

Para revisar la entrega: abrir el repositorio → leer `README.md` → ejecutar `iniciar_api.bat` y `iniciar_gradio.bat` → conversar con el bot → verificar las filas en `reservas.db`.

Gracias · Preguntas